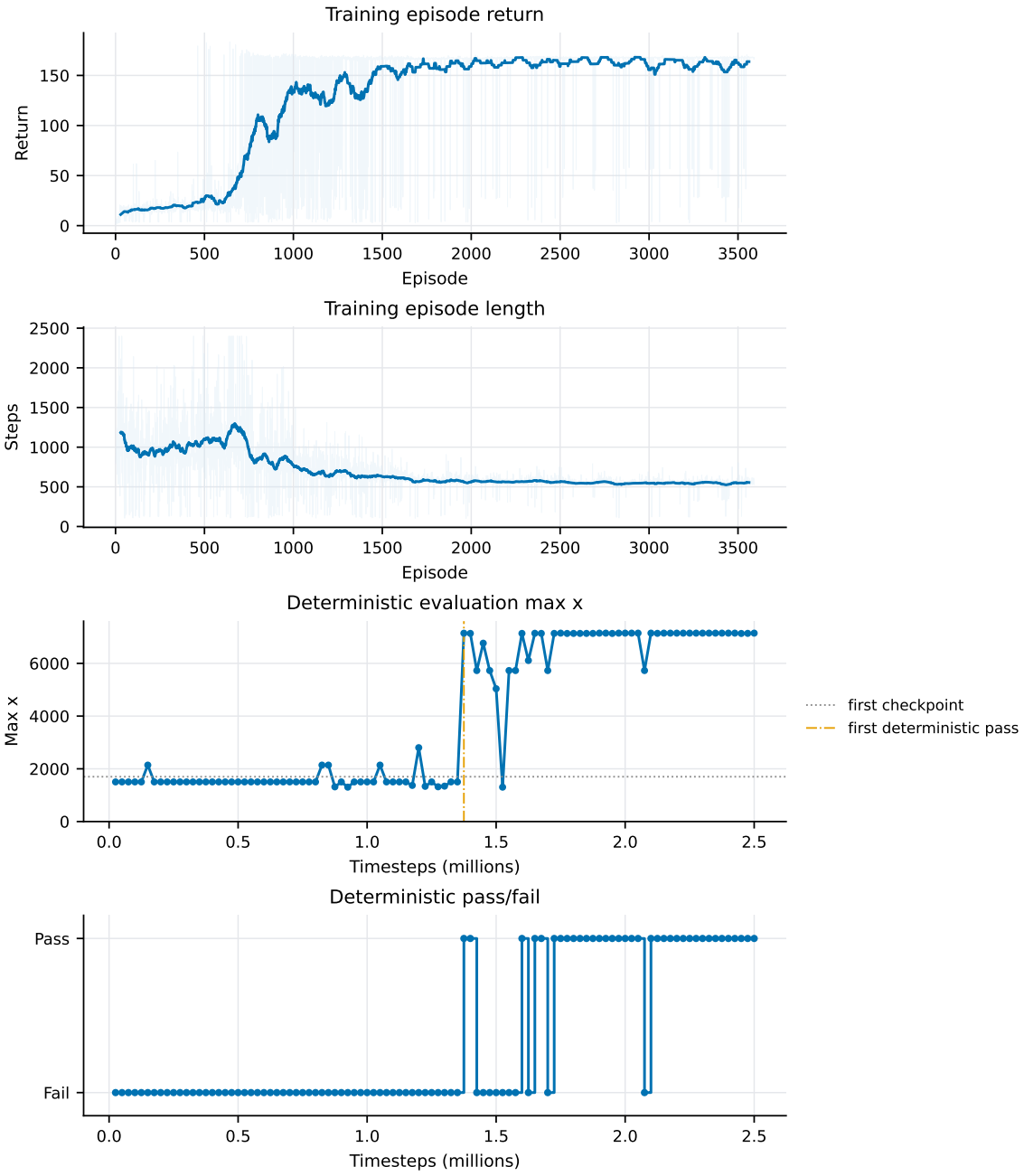
RL Report

Chengyu Yu

May 27, 2026

Training Summary



CNN is Important

An early failed setting flattened the local tile grid and passed it directly to an MLP. In these runs, the policy did not reliably solve the level, suggesting that the flattened grid representation was not enough for this task.

Early setting	Stochastic pass	Deterministic
No CNN, solid grid only	0/50	Fail
No CNN, solid + enemy grid	0/50	Fail

Using a CNN feature extractor for the grid observation improved the result substantially.

Observation Design

The observation design follows the Mario AI Benchmark idea of giving the agent a local receptive field around Mario, represented at block resolution [1]. In my environment this becomes a grid tensor with shape $(\text{channel}, \text{height}, \text{width})$, where channels encode different layout information such as solid tiles, enemies, and shells. This grid is passed through a small CNN instead of being treated as a flat vector, because the important patterns are spatial.

The benchmark also provides Mario state as separate binary or discrete variables, such as whether Mario is on the ground or can jump. I use the same idea for the vector part of the observation: player position, velocity, ground contact, distance to the door, alive state, and a few local ground/wall sensors. These features are already low-dimensional, so they are passed directly to the MLP part of the policy network.

The final observation is therefore a mixed `Dict` observation with a `grid` branch and a `vec` branch, so the SB3 policy type is `MultiInputPolicy`; the grid branch still uses the custom CNN feature extractor.

Fixed Input Shape

Some unused vector-observation dimensions are kept as zero-valued placeholders to keep the input shape fixed across ablations.

Action Design

Engine Patch

I added a small input-adapter patch for jump handling. After a jump has completed, the environment forces the next engine input to release the jump button for one frame. This is needed because continuously sending `jump=true` is not interpreted by the engine as repeated jumps, while a human player can naturally release and press jump again. The patch keeps the learned action interface consistent with the actual human control semantics.

MultiBinary vs. Discrete

Early experiments used a discrete action table, where each action is one complete button combination. These runs showed a strong $x = 1502$ bottleneck, and the final discrete no-left checkpoint

still failed to pass the level.

Action space	Stochastic pass	Deterministic
`MultiBinary(4), final checkpoint`: 4 Bernoulli button decisions	49/50	Pass
`Discrete(8), final checkpoint`: 8 categorical button combinations, no-left	0/50	Fail
`Discrete(16), final checkpoint`: 16 categorical button combinations	30/50	Fail

No-left vs. Full Space

On this map, when keeping the rest of the configuration fixed, the no-left action space performed worse than the full action space.

Reward Design

- I considered two main reward directions. The first uses checkpoint rewards together with a small dense reward for continuous rightward movement. The second removes checkpoint rewards and instead makes the continuous movement reward stronger. In my ablations, the checkpoint-based version performed better.
- The checkpoint reward initially included a human prior. I manually marked positions where the agent often got stuck and placed checkpoints just after those bottlenecks. For example, $x = 1502$ was the first major stuck point, so the first checkpoint was placed at $x = 1700$. Later ablations reduced this prior, leading to the current design: keep the first human-prior checkpoint, then use checkpoints spaced every 1000 pixels.
- The jump-completion bonus is used to encourage useful jumps rather than arbitrary jumping. Since one of the main difficulties in this level is jumping over pits, the reward is only given when a jump results in meaningful forward progress.
- Death and similar penalties are kept relatively small. This level requires exploration, so making failure too expensive would make the policy overly conservative before it has learned how to pass the early bottlenecks.

Hyperparameters

I used a clip range of 0.2. In PPO, this clips the probability ratio inside the surrogate objective to the interval $[0.8, 1.2]$, reducing the incentive for updates that move the new policy too far from the rollout policy [2].

The learning rate was 3×10^{-4}, which is also used in the PPO paper. The discount factor gamma was 0.99, so the agent values long-term progress through the level instead of only immediate local rewards. I also used an entropy coefficient of 0.01, following the paper's use of entropy bonus for exploration.

GAI Disclosure

Codex was responsible for writing all code except `config.py`. For this report, Codex performed simple language improvement and LaTeX formatting.

References

[1] Sergey Karakovskiy and Julian Togelius. The Mario AI Benchmark and Competitions. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(1):55–67, 2012. doi: 10.1109/TCIAIG.2012.2188528.

[2] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017. arXiv:1707.06347.